PROJECT REPORT

PASSIVE-AGGRESSION DETECTION AND CONSTRUCTIVE COMMUNICATION SYSTEM

A Context-Aware Machine Learning Approach for Social Media Analysis and Workplace API Integration

Course: Advanced Python Programming

Role	Name	Registration Number
Primary Developer (Social-Media Application)	Viknesh G	23MID0161
Workplace API Collaborator (Workplace Detection Service)	Abijit I B	23MIC0013

Domain: Machine Learning | Natural Language Processing | Sentiment and Pragmatic Language Analysis | Software Engineering | REST API Integration | Web Application Development

Faculty Guide: Sharmila Banu K

Academic Year: 2026-2027

Project Links

GitHub Repository: https://github.com/viknesh2023-wq/PASSIVE_AGGRESSION_WITH_API

Deployed Application: https://viknesh2023-wq-passive-aggression-streamlit-app-vtj6mf.streamlit.app/unified_router

Demo Video and Presentation (Google Drive):
https://drive.google.com/drive/folders/1elwz1ODxMTES9cOoYEHXepBDUydfO6nO?usp=sharing

Index

Abstract 3
1. Introduction and Project Overview 4
2. Motivation 5
3. Existing Gap and Project Contributions 6
4. System Architecture 7
5. Machine Learning Model Development 8
6. Rule-Based Detection Engine 9
7. Independent Threat Detection 11
8. Constructive Rewrite Assistant 11
9. Dataset Collection and Preparation 12
10. Technology Stack and Software Architecture 13
11. Project Directory Structure 14
12. Development Prompts and Tools 16
13. Context-Aware Query Router 18
14. REST API Integration 19
15. Application Features and User Interface 21
16. Users, Inputs, and Outputs 23
17. Model Evaluation and Results 23
18. Deployment 24
19. Ethical Considerations and Limitations 25
20. Conclusion and Future Scope 26
21. References 26

Abstract

Passive-aggressive communication hides hostility behind polite, positive or neutral wording, which makes it difficult for conventional sentiment analysis to detect. This project presents an English-only system that identifies such covert hostility in written text, explains its reasoning, and suggests constructive alternatives.

The main application, developed by Viknesh G in Streamlit, combines a TF-IDF and Logistic Regression classifier (word 1-3 and character 3-5 n-grams) with a ten-family rule-based engine. Both outputs are preserved, and disagreement between them is flagged for human review. A separate four-class threat classifier works independently of the passive-aggression task, and a rewrite assistant produces meaning-preserving alternatives while refusing to soften genuine threats.

The training corpus described in the project README contains 19,489 records, made of 10,489 deduplicated records from ten source datasets and 9,000 synthetic dialogues, divided into conversation-grouped training, validation and test splits. A context-aware query router directs social-media text to the local pipeline and workplace text to a separately developed FastAPI service built by Abijit I B, which is reached through HTTPX over REST.

As reported in the README, the supervised model reaches 98.70% accuracy and a macro F1 of 0.9872 on 1,382 held-out samples, the hybrid engine 97.68%, and the threat classifier 99.1% accuracy with a macro F1 of 0.963. The rule engine alone scores 40.96% and is valued mainly for interpretable evidence. The report also documents the limitations of the approach and emphasises that predictions should support, not replace, human judgement.

Keywords: passive aggression, natural language processing, TF-IDF, Logistic Regression, rule-based detection, threat detection, Streamlit, FastAPI, REST API.

1. Introduction and Project Overview

1.1 Background

Most people have received a message that looked perfectly polite and still left them feeling slightly stung. "Thanks so much for finally getting back to me" is a good example. Nothing in it is rude, yet nobody reads it as a compliment. This kind of covert hostility is called passive aggression, and it is surprisingly hard for software to notice because the damage is done by the intent behind the words, not by the words themselves.

This project, titled **Passive-Aggression Detection and Constructive Communication System**, is an English-only application that tries to catch that gap. It reads written text, decides whether the message is passive-aggressive, non-passive-aggressive, or too ambiguous to call, and explains why. It also checks independently for threatening language, and it can suggest a calmer, more direct way of saying the same thing. The project sits at the meeting point of Machine Learning, Natural Language Processing, pragmatic language analysis, REST API integration and web application development.

1.2 Project Overview

The main application was built by Viknesh G with a Streamlit frontend. It contains a supervised machine-learning classifier, a rule-based detection engine, an independent threat classifier, a constructive rewrite engine, an SQLite-backed history module and an insights dashboard. A separate workplace passive-aggression model and API service were developed independently by Abijit I B. The two systems talk to each other through a context-aware query router and REST API calls.

One architectural idea runs through the whole report and is worth stating early: both detection systems stay independently maintained. The query router decides which service should handle a message. It does not replace the workplace classifier and it does not run the workplace model's inference. That work happens inside the workplace service, on the machine that hosts it.

1.3 Team and Division of Work

Two students contributed to the overall system, and they did not build the same model. Table 1.1 separates the two contributions so that the ownership of each component is clear.

Table 1.1. Division of work between the two developers

Aspect	Social-Media Application	Workplace Detection Service
Developer	Viknesh G (Reg. No. 23MID0161)	Abijit I B (Reg. No. 23MIC0013)
Repository / project	PASSIVE_AGGRESSION_WITH_API	Separately maintained workplace detection project
Interface	Streamlit multi-page application	FastAPI service run with Uvicorn
Detection logic	TF-IDF + Logistic Regression, rule engine, threat classifier, rewrite engine	Independently trained workplace model
Role in integration	Hosts the router and the HTTPX client	Exposes /health and /predict endpoints
Evaluation reported here	README metrics for the social-media models	Not reported in this document

1.4 Project Objectives

The project was planned around the following ten objectives.

1. Develop an English-only passive-aggression detection system for social-media communication.
2. Implement a supervised machine-learning classifier.

3. Develop a complementary rule-based detection engine.
4. Identify disagreements between the supervised model and the rule-based engine.
5. Independently identify threatening language.
6. Provide constructive rewrites while preserving the original meaning.
7. Support individual comments and multi-message conversation analysis.
8. Develop a context-aware query router.
9. Integrate a separate workplace detection model through REST API communication.
10. Provide analysis history, insights and evaluation information through the Streamlit interface.

1.5 Scope of This Report

The report explains the problem, the design decisions, the data, the models, the integration with the workplace service, the user interface, the reported results and the limits of the work. Wherever a detail could not be confirmed from the project documentation, it is marked **[To be verified]** instead of being guessed. Figures and screenshots are numbered in the order in which they appear.

2. Motivation

2.1 Why Sentiment Analysis Is Not Enough

Classic sentiment analysis sorts text into positive, negative or neutral. That is useful for product reviews, but it breaks down for passive aggression. Take the sentence "Thank you for reminding me of something that was already explained several times." The vocabulary is positive, so a sentiment model would lean positive. A human reader hears indirect criticism. The mismatch between surface wording and communicative intent is exactly what this project sets out to model.

2.2 Patterns the System Looks For

Passive-aggressive writing shows up in several recurring forms, and the system is designed around them:

- **Fake politeness:** courteous phrasing used as cover for irritation.
- **Indirect criticism:** a complaint delivered as an observation or a question.
- **Backhanded compliments:** praise that carries a hidden insult.
- **Implied blame:** wording that points at someone without naming the accusation.
- **Sarcastic agreement:** agreeing in a tone that clearly means the opposite.
- **Dismissiveness:** brushing off a concern while appearing to accept it.
- **Guilt-tripping:** framing the speaker as a quiet martyr to create obligation.

2.3 Why Context and Ambiguity Matter

Short messages such as "Fine." or "Sure, whatever works" can be sincere, tired, or sarcastic. Nobody can reliably tell from the text alone. For this reason the project treats ambiguity as a proper class rather than forcing every message into a yes-or-no answer. The system is allowed to say "this needs more context", and a human then decides.

2.4 Why a Second Domain Was Added

The same idea appears in the workplace, in phrases such as "per my previous email", but the vocabulary and the stakes are different. Instead of retraining one large model on mixed data, the team chose to keep a dedicated workplace model and connect it through an API. That decision keeps each model focused

on one domain and lets each developer maintain, update and redeploy their own component without breaking the other.

2.5 Why Explanations and Rewrites

A label alone is rarely helpful. Someone who is told "passive-aggressive" wants to know which words triggered the flag and what they could say instead. That is why the system highlights matched phrases, gives a rationale, and offers a constructive rewrite. The aim is to help people communicate better, not simply to judge them.

3. Existing Gap and Project Contributions

3.1 The Gap

Widely available text-analysis tools usually address one of three things: overall sentiment, sarcasm, or toxicity. Each captures part of the picture, but passive aggression sits between them. It is rarely toxic in the obvious sense, it is not always sarcastic, and its sentiment is often misleading. Tools that report only a single score also give users little to act on, and they rarely separate genuine threats from harmless figures of speech such as "you are killing it on stage".

Table 3.1. Comparison between a conventional sentiment approach and this project

Concern	Conventional sentiment analysis	This project
Target	Polarity of wording	Covert hostility and intent
Output	Single label or score	Class, rule evidence, highlighted spans, review flag
Ambiguity	Usually forced into a class	Explicit "ambiguous" class
Threats	Often mixed with toxicity	Separate four-class threat classifier
Guidance to user	None	Constructive rewrite suggestions
Domains	One model for all text	Router chooses social-media or workplace service

3.2 Contributions of the Project

- A three-class passive-aggression classifier built from word and character n-gram features.
- A ten-family rule engine that returns matched spans and written evidence.
- A dual-model design that keeps both outputs and flags disagreement with the needs_review mechanism instead of hiding it.
- A threat classifier kept deliberately separate from passive-aggression detection.
- A rewrite engine that preserves meaning and refuses to polish genuinely threatening text.
- A dataset pipeline with manifests, SHA-256 checksums, label mapping, deduplication and conversation-grouped splits.
- A context-aware router and an HTTPX-based integration with an independently built workplace API.
- A seven-page Streamlit application with history, insights and evaluation views.

4. System Architecture

4.1 End-to-End Workflow

Every request begins in the Streamlit application. The text is first preprocessed and checked to make sure it is English. It then reaches the context-aware query router, which decides which detection pipeline should handle it:

```
User Input
   |
   v
Streamlit Application
   |
   v
Preprocessing and English Text Validation
   |
   v
Context-Aware Query Router
   |
   v
Selected Detection Pipeline  ->  Result displayed in Streamlit
```

4.2 Path A: Social-Media Detection

When the router selects the social-media path, the text is handled entirely by components inside the main project. The supervised model and the rule engine run on the same text, and a unified inference layer collects both outputs and compares them. The threat classifier runs separately, the rewrite assistant is available on request, and the result is displayed together with phrase highlighting and evidence.

4.3 Path B: Workplace Detection

When the router selects the workplace path, the main application does not run any workplace model itself. An HTTPX client prepares an HTTP request and sends it to the separate workplace FastAPI service. That service performs its own inference with its own model and returns a structured JSON response, which the Streamlit application parses and displays. A third outcome exists for unclear context: the application can ask for clarification or fall back to an uncertainty-handling path, subject to the final implementation.

4.4 Design Principles

- **Modularity.** Preprocessing, inference, rules, rewriting, history and routing live in separate modules so each can be tested alone.
- **Independence.** The social-media pipeline and the workplace service can be updated, redeployed or replaced without editing each other.
- **Transparency.** Where two detectors disagree, both outputs are preserved and the case is marked for human review.
- **Separation of concerns.** Domain routing, passive-aggression classification and threat classification are three different problems and are handled separately.

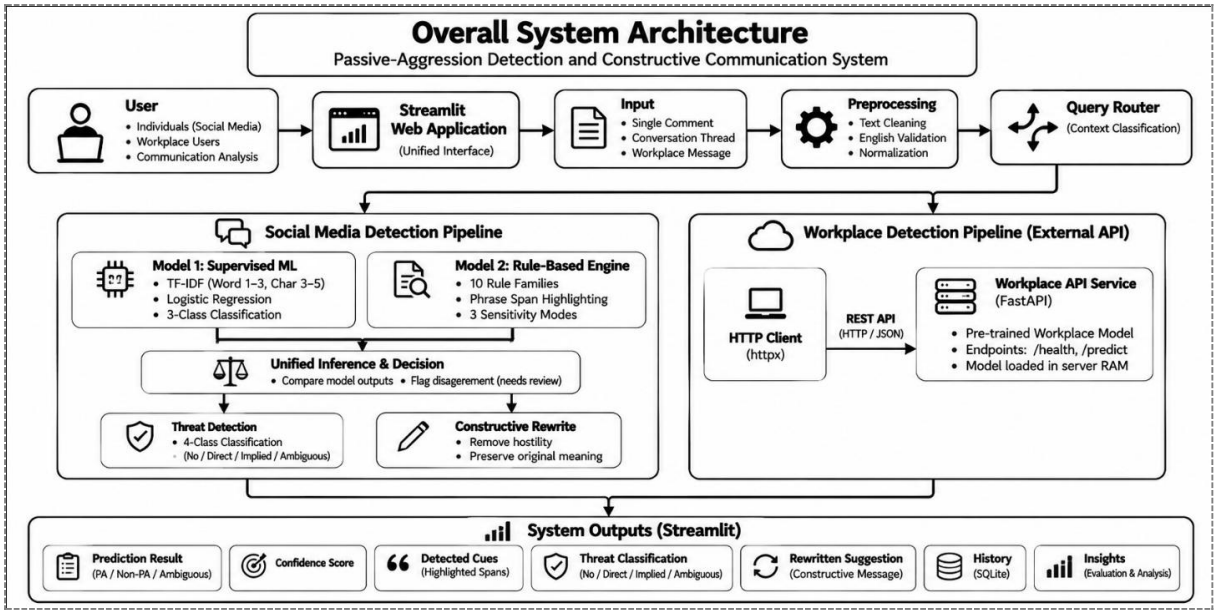


Figure 1. ***Overall system architecture.*** Overall architecture of the context-aware passive-aggression detection system, showing the router, the two detection paths and the shared Streamlit interface.

Besides detection, the application also provides history, insights, conversation analysis, rewriting and an About page. These are covered in Chapter 15.

5. Machine Learning Model Development

5.1 Model Type

The supervised component is a classical machine-learning pipeline built with Scikit-learn: TF-IDF feature extraction followed by Logistic Regression. The vectoriser uses **word n-grams from 1 to 3** and **character n-grams from 3 to 5**, and the classifier assigns each text to one of three passive-aggression classes. A linear model on sparse text features was chosen because it trains quickly, runs in a few milliseconds per sample and stays reasonably easy to reason about.

5.2 Classification Labels

Table 5.1. Three-class passive-aggression labels

Label	Meaning
passive_aggressive	Indirect hostility, fake politeness, backhanded compliments, implied blame or similar covert hostility
non_passive_aggressive	Neutral information, sincere praise, constructive feedback or direct non-hostile communication
ambiguous	Short or unclear statements that need additional human context

5.3 How the Model Processes Text

1. The text is collected from the Streamlit interface.
2. Preprocessing and English-language validation are applied.
3. TF-IDF converts the text into a numerical feature vector.
4. Word and character n-grams capture different kinds of linguistic pattern.
5. Logistic Regression calculates the class decision.
6. The result is returned to the inference layer, which handles comparison with the rule engine.

5.4 Why Both Word and Character N-grams

Each type of n-gram notices something the other misses. Word n-grams of length one to three capture phrase-level habits, such as polite openers combined with a complaint. Character n-grams of length three to five work below the word level, so they tolerate spelling variations, stretched words, informal spellings and punctuation habits that are common on social media. Used together, they give the classifier both the "what was said" and the "how it was typed" view of the same sentence.

5.5 Training and Evaluation Protocol

The model was trained on the processed training split and evaluated on the held-out test split, with splits grouped by conversation so that replies from one conversation never appear on both sides of a split. The evaluation reports accuracy, macro F1, weighted F1 and inference latency per sample. The trained pipeline is saved as a reusable artifact and loaded by the inference layer. The supplied documentation does not list the exact training hyperparameters, so this report does not state them; they should be read from the training scripts in the repository **[To be verified]**.

5.6 Model Artifacts

Serialized artifacts are kept in the models directory. The documented files include pa_classifier_v2.pkl, which holds the current supervised pipeline, threat_classifier_v1.pkl for the threat task, and pa_classifier.pkl, which appears to be an earlier version retained in the repository. The older Legacy LinearSVM variant shown in the evaluation table is reported only for comparison.

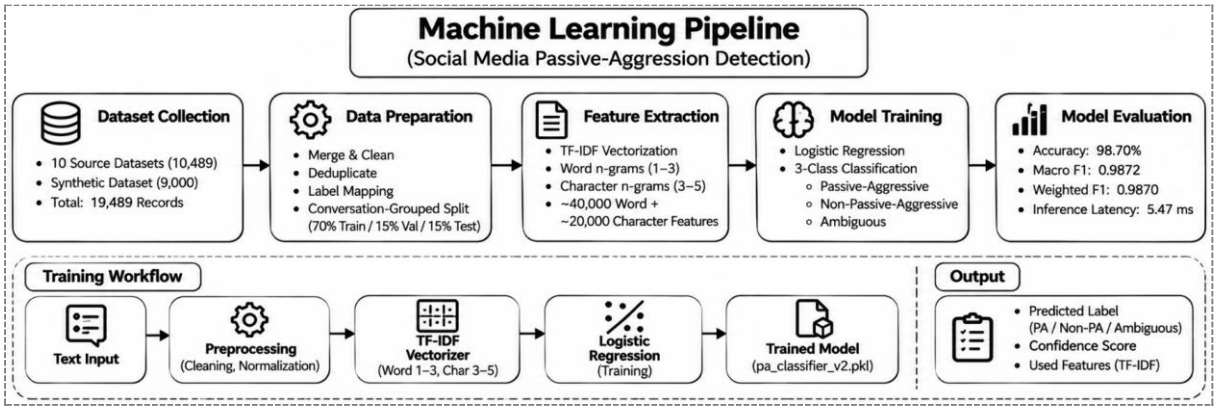


Figure 2. ***Machine-learning pipeline.*** Text preprocessing, TF-IDF with word and character n-grams, and Logistic Regression classification.

6. Rule-Based Detection Engine

6.1 Purpose

The second detector is a rule engine. Where the machine-learning model gives a prediction, the rule engine gives a reason. It matches linguistic patterns, returns the exact character spans that triggered a rule, and adds a short written rationale. That makes it the main source of phrase highlighting and interpretable evidence in the interface.

6.2 The Ten Rule Families

Table 6.1. Rule families in the detection engine

No.	Rule family	What it targets
1	Fake politeness	Courtesy phrases used to carry irritation
2	Indirect criticism	Complaints phrased as observations or questions
3	Backhanded compliments	Praise that contains a hidden insult

No.	Rule family	What it targets
4	Implied blame	Responsibility assigned without a direct accusation
5	Sarcastic agreement	Agreement that signals the opposite
6	Dismissiveness	Brushing off a concern or the person raising it
7	Guilt-tripping / martyrdom	Making the other person feel obliged or responsible
8	Contextual escalation	Using the parent comment to raise the reading of a reply
9	Benign overrides	Cancelling a match when the phrase is harmless in context
10	Quotation, negation and figurative language	Avoiding flags on quoted, negated or figurative wording

6.3 Outputs of the Rule Engine

For every analysed text, the engine can return the matched text spans, the family that fired, and an evidence string explaining the match. The Streamlit interface uses the spans to highlight phrases directly inside the original comment. Benign overrides exist because an engine based only on patterns would otherwise flag a lot of ordinary speech, so they are an important part of keeping false alarms under control.

6.4 Sensitivity Modes

Table 6.2. Sensitivity modes of the rule engine

Mode	Purpose
balanced	Default balance between precision and recall
strict	Higher recall and heightened sensitivity
lenient	Higher precision and fewer low-confidence flags

6.5 Disagreement Handling

When the supervised model and the rule engine give different predictions, the application does not quietly pick one. Both outputs are kept, shown side by side, and the case is flagged through the needs_review mechanism. This matters because the two detectors fail in different ways, and a disagreement is often the most informative signal that a human should look at the message.

6.6 Role and Limits

The rule engine is not presented as being as accurate as the supervised classifier. In the reported evaluation it scores far lower on the held-out test set (Chapter 17). Its value lies in interpretable evidence and phrase-level highlighting, not in standalone accuracy, and that distinction should be kept in mind when reading the results.

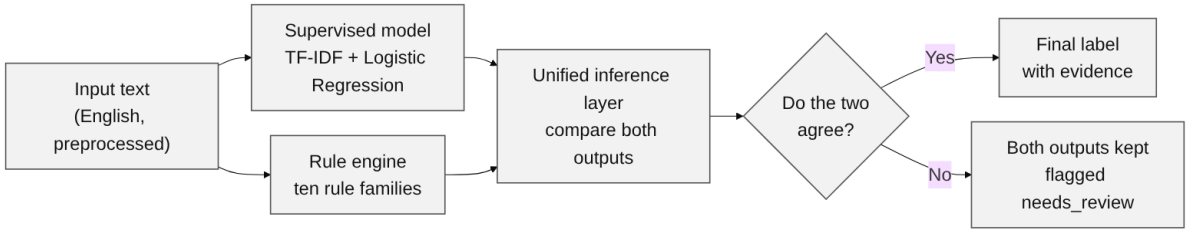


Figure 3. ***Dual-model detection workflow.*** The supervised model and rule engine run independently, are compared by the inference layer, and disagreements are marked needs_review.

7. Independent Threat Detection

7.1 Why Threats Are a Separate Task

Passive aggression is indirect hostility. A threat is about harm, whether stated outright or implied. A message can be one without the other: a threat can be written in perfectly blunt language with no passive-aggressive tone at all, and a passive-aggressive message can be entirely harmless in terms of safety. Combining the two into one label would blur both, so the threat classifier operates on its own and produces its own result.

7.2 Threat Classes

Table 7.1. Four-class threat labels

Threat class	Explanation
no_threat	Ordinary communication and harmless figurative language
direct_threat	Explicit physical harm, violence, tracking or destruction threats
implied_threat	Indirect conditional harm or intimidation
ambiguous	Statements that need additional safety review

7.3 Telling Real Threats from Figures of Speech

A large part of the difficulty is that English is full of violent idioms. "You are killing it on stage" is praise, while "I will track you down and hurt you" is a direct threat. The classifier is built to separate these, including gaming banter and competitive idioms, so that ordinary conversation is not flagged as dangerous.

7.4 Reported Performance

The project README reports an accuracy of **99.1%** and a macro F1 of **0.963** for the threat classifier. These are the held-out evaluation results reported in the project documentation and were not independently re-run for this report. They belong to the threat task only and must not be combined with the passive-aggression metrics.

8. Constructive Rewrite Assistant

8.1 Purpose

The rewrite assistant offers alternative wording that lowers the covert hostility in a message while keeping what the writer actually wanted to say. If a person is annoyed that a task was repeated, the rewrite should still say so, only directly and without the sting.

8.2 Design Principles

- Preserve the core meaning and the original concern.
- Remove indirect hostility.
- Do not invent facts that were not in the original.
- Do not add accusations that were not in the original.
- Do not create new commitments on the writer's behalf.
- Offer direct, constructive wording as an alternative.

8.3 Illustration of the Intended Style

The following pair illustrates the style the assistant is aiming for. It is an explanatory example written for this report, not a recorded output of the system.

Table 8.1. Illustrative original and constructive version

Original (passive-aggressive)	Constructive version (illustrative)
"Thank you for reminding me of something that was already explained several times."	"I think we covered this earlier. Could we check the earlier explanation together so we are on the same page?"

8.4 Safety Handling

Threatening language is never turned into a polite request. If the original text contains a threat, the application returns a safety warning or refusal instead of a softened version, because a polite rewrite of a threat would make it look acceptable. The rewrite is only a recommendation; the user always keeps the choice to send the original wording.

9. Dataset Collection and Preparation

9.1 Source Datasets

According to the project README, the corpus is made of ten source datasets holding 10,489 records and one separate synthetic dataset holding 9,000 records, giving 19,489 records before any training-specific filtering. Table 9.1 lists them.

Table 9.1. Datasets used in the project

No.	Dataset	Records	Purpose / role
1	tweeteval_sentiment	3,500	Sentiment and non-PA baseline examples
2	sarc_reddit_regular	2,000	Regular Reddit text
3	sarc_reddit_sarcasm	2,000	Sarcasm-related auxiliary examples
4	goemotions	1,000	Emotion-related examples
5	goemotions_anger_hostility	883	Anger and overt-hostility examples
6	genz_sarc_banter	800	Informal social-media language
7	authentic_pa_mined	143	Mined passive-aggression examples
8	curated_pa_tactics	53	Curated passive-aggression tactic examples
9	contrastive_hard_negatives	60	Hard-negative examples
10	gold_pa_anchors	50	Annotator-verified anchor examples
11	SYNPA synthetic dataset	9,000	Synthetic dialogue examples
	Total	**19,489**	10,489 source + 9,000 synthetic

9.2 Primary and Auxiliary Sources

It would be wrong to read this table as ten collections of passive-aggression examples. Only some of the sources, such as the mined, curated and gold anchor sets and the synthetic dialogues, were built around passive aggression. The sentiment, sarcasm and emotion collections act as auxiliary or contrastive material that supplies ordinary, hostile or sarcastic text for the model to learn the difference from. The final label mapping decides which records are usable for each training task.

9.3 Count Reconciliation

The README documents a small discrepancy that we have kept visible instead of quietly adjusting.

Table 9.2. Corpus count reconciliation as documented

Item	Value
Expected rows	10,520
Deduplicated rows	10,489
Difference	31 records
Removed: near-duplicate tactic expansion templates	29
Removed: unannotated placeholder rows from the gold anchor source	2

9.4 Data Preparation Steps

- **Local organisation:** raw sources are kept under data/raw/existing_sources and data/raw/synthetic.
- **Schema auditing:** each source was inspected and standardised to a consistent schema.
- **Manifests and checksums:** dataset_manifest.json and source_checksums.csv record what was used, and SHA-256 checksums make it possible to detect any later change to a source file.
- **Label mapping:** label_mapping.csv defines how each source label maps onto the project classes.
- **Deduplication:** exact and near-duplicate records were removed.
- **Synthetic data storage:** synthetic dialogues were produced in manageable batches and audited for duplicates before being merged.

9.5 Leakage Prevention and Splitting

Data is divided into **70% training, 15% validation and 15% testing**. The split is grouped by conversation_id, so that every message from a given conversation falls in the same split. Without this, near-identical replies from one thread could appear in both training and test data and inflate the scores. The held-out test file is data/processed/pa_test.csv, and the README reports 1,382 conversation-grouped samples in that evaluation summary.

Table 9.3. Data splits

Split	Share	File
Training	70%	data/processed/pa_train.csv
Validation	15%	data/processed/pa_val.csv
Test	15%	data/processed/pa_test.csv (1,382 samples reported)

10. Technology Stack and Software Architecture

10.1 Technologies Used

Table 10.1 lists the technologies that appear in the project materials and the job each one does. Entries marked as deployment options were considered or identified for deployment; they should not be read as proof of a live production setup.

Table 10.1. Technology stack

Technology	Purpose	Remarks
Python	Main programming language	Used across both systems

Technology	Purpose	Remarks
Streamlit	User-facing frontend	Active frontend of the main application
Scikit-learn	Machine-learning implementation	Pipeline, vectoriser and classifier
TF-IDF Vectorizer	Numerical text features	Word 1-3 and character 3-5 n-grams
Logistic Regression	Three-class passive-aggression classifier	Supervised component
Rule-based / Regex engine	Pattern detection and phrase highlighting	Ten rule families
FastAPI	Workplace REST API backend	Workplace service
Uvicorn	ASGI server	Runs the FastAPI service
HTTPX	HTTP communication between the systems	Client side in Streamlit app
SQLite	Persistent analysis history	Database file under backend/
Joblib / Pickle	Model serialization and storage	.pkl artifacts in models/
GitHub	Source-code management	Repository hosting
Docker	Containerisation option for the workplace API	Deployment option
Render	Cloud deployment option	Identified in project materials; live use not confirmed
Gemini API	Context routing, where configured	Domain classification only, not PA detection

10.2 Frontend Clarification

Streamlit is the active user-facing frontend. Older React/Vite files may still exist in the wider repository, but they are not treated as the active interface in this report **[To be verified against the repository]**.

10.3 Software Architecture in Layers

Table 10.2. Layered view of the software

Layer	Responsibility	Main files / components
Presentation	Pages, forms, highlighting, charts	streamlit_app.py, pages/
Routing	Choose social-media or workplace analysis	Context router, HTTPX client
Inference	Preprocess, run models, compare results	preprocess.py, pa_inference.py, rule_detector.py
Assistance	Constructive rewriting and safety refusal	rewrite_engine.py
Persistence	Store analysis history	history_db.py, pa_detector.db
Model storage	Serialized classifiers	models/*.pkl
External service	Workplace inference	workplace_api/ (separate service)

11. Project Directory Structure

11.1 Directory Tree

The repository is organised as follows. The structure is reproduced from the project documentation and no extra files have been added to it.

```
PASSIVE_AGGRESSION_WITH_API/
|-- .streamlit/            config.toml
|-- backend/               pa_detector.db, tests/
|-- data/
```

```
|   |-- manifests/          dataset_manifest.json, source_checksums.csv,
|   |                       label_mapping.csv
|   |-- processed/          pa_train.csv, pa_val.csv, pa_test.csv
|   `-- raw/                existing_sources/, synthetic/
|-- inference/              history_db.py, pa_inference.py, preprocess.py,
|                           rewrite_engine.py, rule_detector.py
|-- models/                 pa_classifier_v2.pkl, threat_classifier_v1.pkl,
|                           pa_classifier.pkl
|-- pages/                  1_analyze_comment.py, 2_analyze_conversation.py,
|                           3_rewrite_comment.py, 4_history.py,
|                           5_insights.py, 6_about.py
|-- reports/
|-- tests/
|-- training/
|-- workplace_api/          main.py
|-- requirements.txt
|-- README.md
`-- streamlit_app.py
```

11.2 Responsibility of Important Files

Table 11.1. Important files and what they do

File / folder	Responsibility
streamlit_app.py	Entry point and home page: overview, architecture, taxonomy, model status
pages/1_analyze_comment.py	Single-comment analysis with dual-model comparison
pages/2_analyze_conversation.py	Multi-message analysis with parent-comment context
pages/3_rewrite_comment.py	Constructive rewrite interface
pages/4_history.py	Saved analyses, filters, search and CSV export
pages/5_insights.py	Evaluation metrics, confusion matrices, provenance charts
pages/6_about.py	Background, taxonomy, ethics and deployment notes
inference/preprocess.py	Text cleaning and English-language validation
inference/pa_inference.py	Unified inference that runs both detectors and compares them
inference/rule_detector.py	Rule families, matched spans, sensitivity modes
inference/rewrite_engine.py	Meaning-preserving rewrites and safety refusal
inference/history_db.py	SQLite history storage and retrieval
models/	Serialized supervised and threat classifier artifacts
data/manifests/	Dataset manifest, source checksums, label mapping
data/processed/	Train, validation and test CSV files
workplace_api/main.py	Reference to the workplace FastAPI application
training/, tests/, reports/	Training scripts, regression tests and generated evaluation reports
requirements.txt	Python dependencies

Two points need checking against the repository. First, the directory listing does not show a separate router page, yet the deployed URL ends in /unified_router, so the router page or module should be identified by its real filename **[To be verified]**. Second, the location of the workplace model artifact inside the workplace service is not stated in the supplied material **[To be verified]**.

12. Development Prompts and Tools

12.1 Representative Development Prompts Used During Project Construction

The project was built in stages, and each stage was driven by a clear instruction describing what the component had to do. Table 12.1 records sixteen of the principal development prompts, grouped by phase. They are **representative reconstructions** of the main development instructions and objectives. They are not verbatim exports of the original conversation history, and the "expected output" column describes what each prompt was intended to produce, not a guarantee of how any single run turned out.

Table 12.1. Representative development prompts

ID	Phase	Objective	Prompt used	Expected development output
P01	Project Design	Initial project planning	Develop an English-only passive-aggression detection system for social-media communication. The system should identify indirect criticism, fake politeness, backhanded compliments, sarcastic agreement, implied blame, and dismissive language. Design the application with a clear machine-learning workflow, explainable predictions, and a user-friendly interface. Keep the architecture modular so that additional detection components can be integrated later.	Initial project structure, objectives, detection workflow and development plan
P02	Data Engineering	Dataset integration and preparation	Inspect the available social-media, sarcasm, sentiment, and emotion datasets. Integrate the sources into the project directory, maintain dataset provenance, identify duplicates, standardise the schema, and prepare a consistent English-only dataset. Preserve the distinction between primary passive-aggression labels and auxiliary sentiment, sarcasm, or emotion labels. Generate dataset manifests and leakage-resistant conversation-grouped training, validation, and test splits.	Consolidated dataset structure, manifests, label mapping and splits
P03	Data Augmentation	Synthetic dataset construction	Create a structured synthetic English social-media dialogue dataset containing realistic passive-aggressive, non-passive-aggressive, and ambiguous examples. Include varied writing styles, informal language, sarcasm, indirect criticism, mock politeness, implied blame, and neutral hard negatives. Maintain consistent labels, avoid repetitive templates, and preserve unique record identifiers. Save the data in manageable batches and audit duplicates before merging.	Structured synthetic dialogue records and audit information
P04	Model Development	Supervised machine-learning model	Develop a three-class passive-aggression classifier using Scikit-learn. Use TF-IDF feature extraction with word n-grams and character n-grams followed by Logistic Regression. Train and evaluate the model using conversation-grouped data splits. Report accuracy, macro F1, weighted F1, and inference latency. Save the trained pipeline as a reusable model artifact and provide a prediction interface.	Trained classifier, evaluation results, serialized artifact
P05	Explainable Detection	Rule-based detection engine	Create an independent rule-based passive-aggression detection engine with clearly separated rule families. Cover fake politeness, indirect criticism, backhanded compliments, implied blame, sarcastic agreement, dismissiveness, guilt-tripping, contextual escalation, benign overrides, and quotation or negation handling. Return matched character spans and explanatory evidence. Support balanced, strict, and lenient sensitivity modes.	Rule detector, phrase highlighting, sensitivity modes, explanations
P06	Inference Orchestration	Dual-model comparison and integration	Integrate the supervised Logistic Regression model and rule-based detector through a unified inference layer. Execute both detection components independently, preserve their individual outputs, and expose their comparison. When predictions disagree, flag the case for human review instead of silently allowing one model to overwrite the other. Keep the inference logic modular and testable.	Unified inference orchestrator and disagreement handling

ID	Phase	Objective	Prompt used	Expected development output
P07	Safety Classification	Independent threat detection	Implement a separate four-class threat detection component with the labels no_threat, direct_threat, implied_threat, and ambiguous. Distinguish actual threatening language from harmless figurative expressions, gaming banter, and ordinary competitive idioms. Keep threat classification independent of the passive-aggression prediction and expose a separate result.	Independent four-class threat classifier
P08	Communication Assistance	Constructive rewrite engine	Develop a constructive rewriting assistant for passive-aggressive messages. The rewritten message should remove covert hostility while preserving the user's underlying concern and core meaning. Do not invent facts, introduce accusations, or add commitments that were not present in the original. If the original text contains threatening language, return a safety refusal instead of converting it into a polite request.	Meaning-preserving rewrite engine and safety refusal
P09	Frontend Development	Streamlit application development	Create a professional multi-page Streamlit application for passive-aggression detection. Include pages for home, comment analysis, conversation analysis, constructive rewriting, history, insights, and about. Use a pure black theme with charcoal panels, white text, and restrained red accents. Display model predictions, rule evidence, highlighted phrases, sensitivity settings, and relevant analysis information in a clear layout.	Seven-page Streamlit application
P10	Multi-Domain Routing	Context-aware query router	Implement a context-aware query router that identifies whether an incoming message belongs to social-media communication or workplace communication. Route social-media input to the locally available social-media detection pipeline and workplace input to the separate workplace REST API. Handle uncertain context appropriately. Keep domain routing separate from passive-aggression classification and preserve the output of the selected detection system.	Unified query router and domain-based dispatch
P11	Backend API Integration	Workplace REST API development	Create a dedicated FastAPI backend service around the independently trained workplace passive-aggression model. Load the serialized model artifact from the configured model directory during service startup. Implement GET /health for service readiness and POST /predict for text classification. Validate incoming JSON, perform inference using the loaded pipeline, and return a structured JSON response. Keep the API service independent from the social-media application.	FastAPI microservice with model loading, validation and endpoints
P12	Cross-System Communication	HTTPX client integration	Integrate the workplace FastAPI service into the main Streamlit application using Python HTTPX. Configure the API base URL through environment variables or application secrets. Add a health check, prediction request, network timeout, response parsing, and connection-error handling. Preserve the original workplace API response fields and clearly identify the source of the result.	HTTP client dispatcher and workplace API integration
P13	Data Persistence	History and SQLite persistence	Implement persistent analysis history using SQLite. Store the supported analysis records and provide filtering by prediction label, keyword search, detailed record viewing, summary statistics, and CSV export. Keep the history module separate from the detection logic.	SQLite integration and history interface
P14	Analytics	Insights and evaluation dashboard	Develop an insights page that displays the actual offline evaluation metrics from the project's held-out test results. Include suitable charts, confusion matrices, dataset provenance information, and error breakdowns. Ensure that the dashboard reads actual project evaluation data rather than using invented or demonstration metrics.	Insights dashboard and evaluation visualisations

ID	Phase	Objective	Prompt used	Expected development output
P15	Quality Assurance	Testing and regression validation	Create an automated regression test suite covering the passive-aggression classifier, rule-based detector, dual-model inference, threat classifier, constructive rewrite engine, and application integration. Test normal, passive-aggressive, ambiguous, threatening, and figurative-language examples. Verify that API failures are handled appropriately and that model disagreements are preserved.	Automated tests, regression checks and test reports
P16	Deployment	Deployment and final integration	Prepare the Streamlit application for deployment. Verify that the project uses its internal data, model artifacts, configuration, and dependencies correctly. Configure required secrets through the deployment platform rather than hardcoding credentials. Verify the context router, workplace API connection, history functionality, and main application pages. Document any external service dependency and deployment limitation.	Deployment-ready application and validation notes

12.2 Note on the Prompts

The prompts presented in this section are representative reconstructions of the main development instructions used to describe the project's implementation workflow. They document the objectives and development activities associated with the project. They are not claimed to be verbatim exports of the complete original AI conversation history.

12.3 Tools

Development relied on Python and its scientific stack, Scikit-learn for modelling, Streamlit for the interface, FastAPI and Uvicorn for the workplace service, HTTPX for client calls, SQLite for history, Git and GitHub for version control, and Docker as a containerisation option for the workplace API. AI assistance was used as a development aid in line with the prompts above; the design decisions, data choices and evaluation reporting remain the responsibility of the developers.

13. Context-Aware Query Router

13.1 Responsibility

The router has one job: work out what kind of communication it has been given and choose the right destination. It is not a passive-aggression classifier. It never decides whether a message is hostile, only which system is the right one to ask.

13.2 Possible Routes

Table 13.1. Routes selected by the router

Route	Destination
social_media	Main social-media detection pipeline (local to the Streamlit project)
workplace	Separate workplace REST API
uncertain	Clarification or uncertainty-handling path, if implemented

13.3 Routing Flow

```
User enters text
   |
Context router evaluates the communication context
   |
Social media  or  Workplace  path is selected
   |
```

```
Appropriate detector processes the text
   |
Result is displayed in the main application
```

13.4 Role of Gemini

Where the Gemini API is configured for domain routing, its purpose is context classification and service selection only. It does not detect passive aggression. The social-media pipeline does that for social-media text, and the workplace model does that for workplace text after the request reaches the workplace API. Whether Gemini is active in the final deployed version should be confirmed**.**

13.5 Why Routing Is Kept Separate

Keeping routing independent from classification means either detector can be improved without touching the router, and the router can be tested with simple domain examples. It also guarantees that the output the user sees comes from the detector that was actually selected, with its own fields intact.

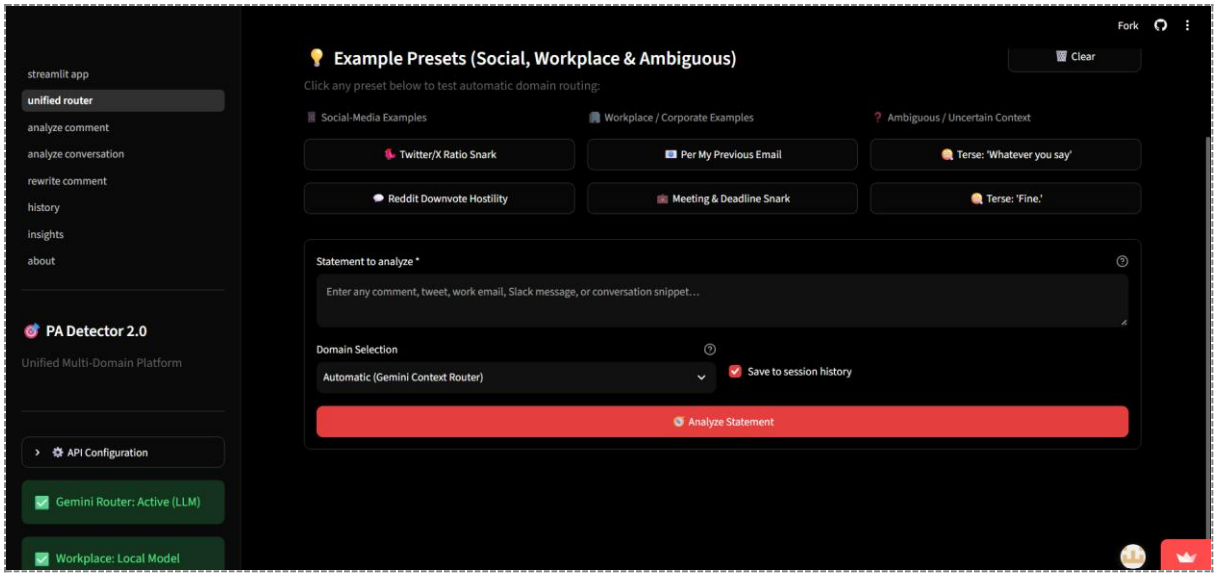


Figure 4. ***Query router screenshot.*** The Unified Router page showing the domain decision and the selected detection path.

14. REST API Integration

14.1 The Two Systems

The most important thing to understand about the integration is that two independently maintained programs cooperate over the network. Neither one imports the other's code.

Table 14.1. The two cooperating systems

	Social-media application	**Workplace detection service**
Developer	Viknesh G	Abijit I B
Framework	Streamlit	FastAPI served by Uvicorn
Communication role	Client: sends requests with HTTPX	Server: exposes /health and /predict
Port mentioned	-	8001 (from the presentation)
Model	Social-media classifiers (local)	Workplace model (on the API host)

14.2 Step 1: Workplace Model and API Backend

The workplace model is trained and maintained separately. It is made available to the API backend, and the API service loads the trained model from its configured storage location. The model artifact and any API authentication credential are two separate things. A key authorises a caller; it does not contain a model.

14.3 Step 2: Model Storage

Storage happens at two levels. In **persistent storage**, the serialized model file sits on the disk of the server that hosts the workplace API. At **runtime**, as described in the presentation, the service loads the model into RAM when it starts. This avoids reading the file from disk for every prediction, which makes responses quicker. It does not make them instant, since network travel and inference still take time.

14.4 Step 3: Endpoints

Table 14.2. Workplace API endpoints

Endpoint	Purpose
GET /health	Checks whether the API is reachable, reports model readiness where supported, and returns service status
POST /predict	Receives a text statement, validates the request, runs workplace inference and returns structured JSON

14.5 Step 4: Client Request

The Streamlit application uses HTTPX to send the workplace text to the API, for example:

```
{
  "text": "Per my previous email, please review."
}
```

14.6 Step 5: Server Processing

1. The workplace API receives the request.
2. It validates the JSON.
3. It passes the text to the loaded workplace model pipeline.
4. It performs classification.
5. It builds the prediction metadata that the service supports.

14.7 Step 6: Response

The service replies with JSON. The payload below is purely illustrative and is **not** a verified experimental result. The score of 0.948 is an example value, and the real field names must be taken from the actual workplace API implementation **[To be verified]**.

```
{
  "label": "Potentially passive-aggressive",
  "score": 0.948,
  "cues": ["previous email"]
}
```

14.8 Step 7: Display

The main application parses the response and shows it in the Streamlit interface, labelled so that the user can see it came from the workplace service and not from the social-media models.

14.9 Step 8: Error Handling

The presentation describes an HTTP timeout and an offline fallback concept. The client is meant to stop waiting after a set time, report a clear message when the service cannot be reached, and avoid crashing the page. Whether the fallback itself is operational depends on the actual code and has not been confirmed here **[To be verified]**.

14.10 Where the Model Really Runs

The workplace model executes on the machine or cloud service that hosts the workplace API. It does not run on the computer of the social-media application merely because that application holds an API key or knows the service address. The Streamlit side only sends text and receives a reply.

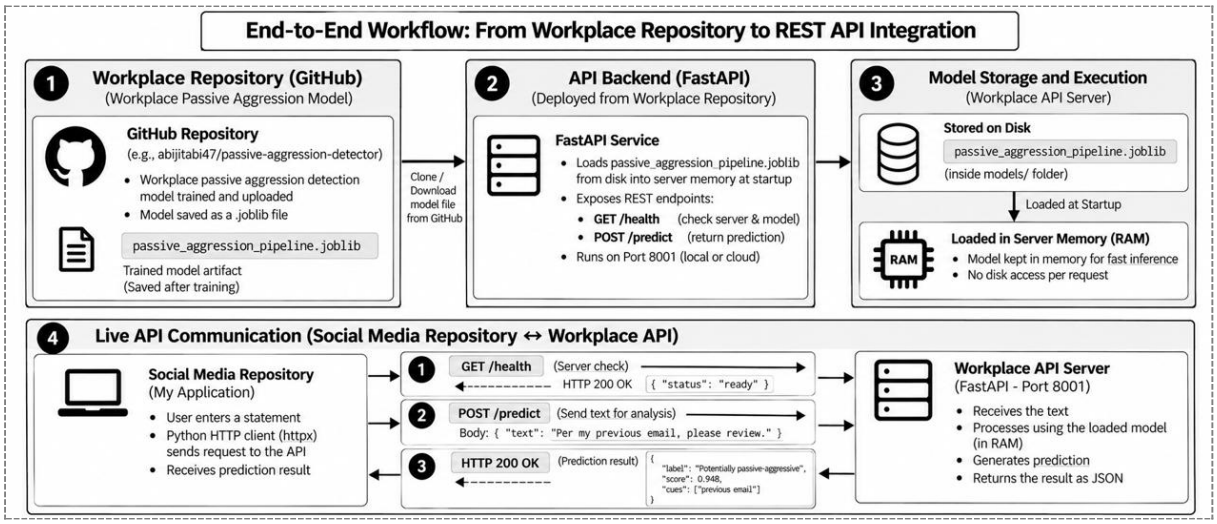


Figure 5. ***Workplace API architecture.*** Streamlit client, HTTPX request, FastAPI service and JSON response.

15. Application Features and User Interface

The README documents a seven-page Streamlit application. The interface follows a pure black theme with charcoal panels, white text and restrained red accents, as set in the development prompts.

15.1 Home Page

Implemented in streamlit_app.py. It gives a system overview, architecture information, the classification taxonomy and the status of the loaded models.

15.2 Analyze Comment

Implemented in pages/1_analyze_comment.py. It handles single-comment analysis and shows the supervised prediction, the rule-engine result, the dual-model comparison and phrase highlighting. The user can select a sensitivity mode and can jump to the rewrite action from the result.

15.3 Analyze Conversation

Implemented in pages/2_analyze_conversation.py. It accepts a multi-message thread, treats earlier messages as parent context, analyses the target reply, and applies contextual escalation so that a reply to a hostile parent is read in that light.

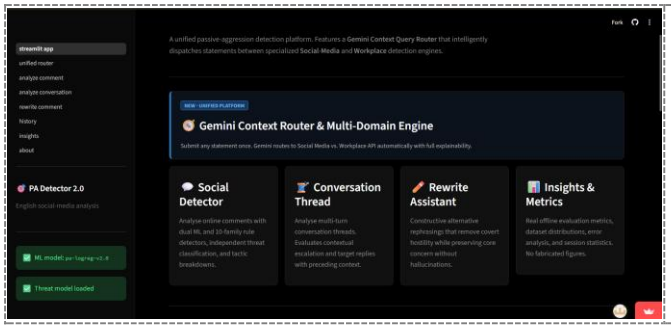


Figure 6. ***Home page.*** Overview, taxonomy and model status.

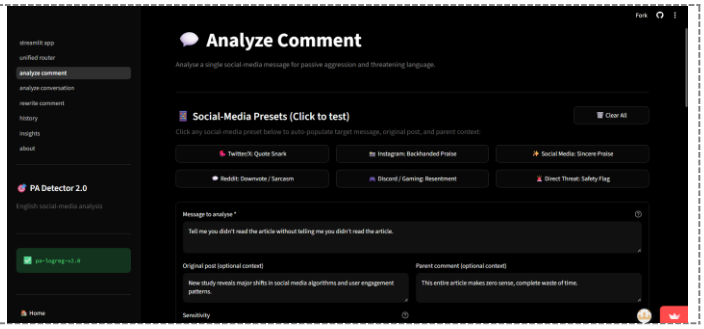


Figure 7. ***Social-media comment analysis.*** Dual-model result with highlighted phrases.

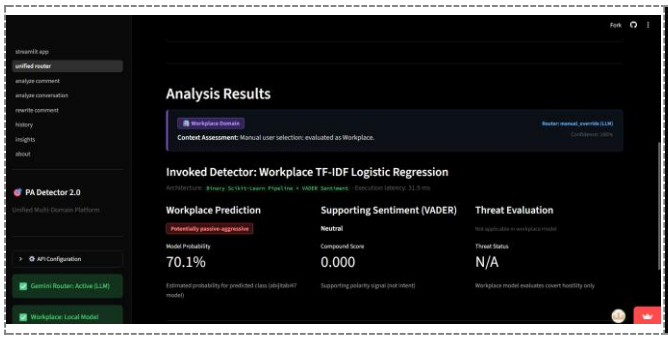


Figure 8. ***Workplace analysis.*** Result returned by the workplace service.

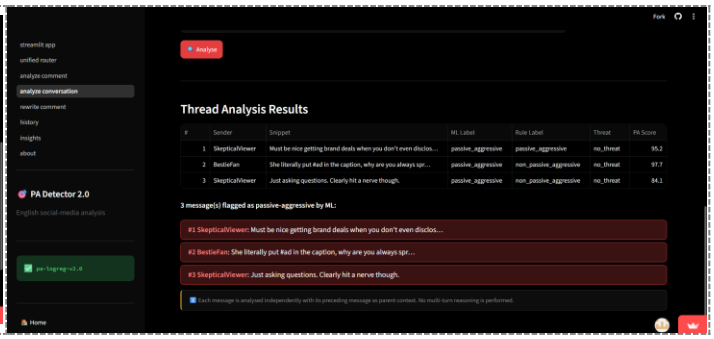


Figure 9. ***Conversation analysis.*** Parent comment and target reply analysis.

15.4 Rewrite Comment

Implemented in pages/3_rewrite_comment.py. It offers constructive rephrasing that preserves meaning, gives several alternative suggestions and supports one-click copying. The suggested rewrites follow the approach described in Chapter 8.

15.5 History

Implemented in pages/4_history.py. Analysis records are stored in SQLite and can be filtered by label, searched by keyword, opened in a detailed view and exported to CSV.

15.6 Insights

Implemented in pages/5_insights.py. It presents the offline evaluation metrics, confusion matrices, dataset provenance charts and error breakdowns, based on the project's held-out evaluation data and not on demonstration numbers.

15.7 About

Implemented in pages/6_about.py. It covers the research background, classification taxonomy, ethical disclaimers and deployment specifications.

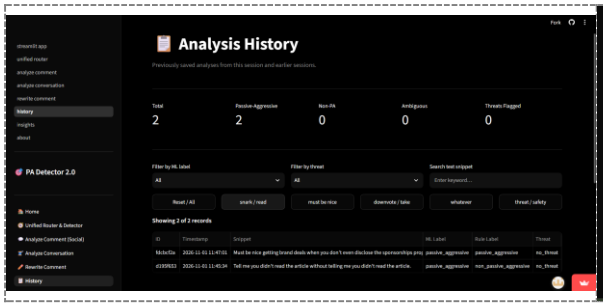


Figure 10. ***History page.*** Saved analyses with filters and export.

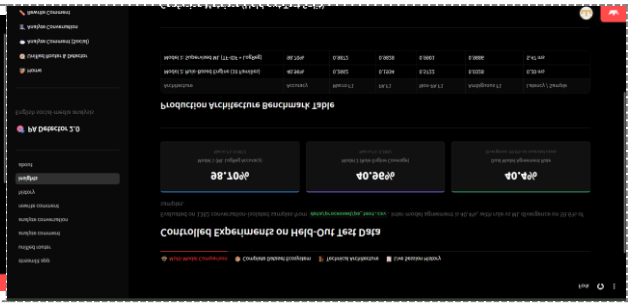


Figure 11. ***Insights dashboard.*** Metrics, confusion matrices and provenance.

16. Users, Inputs, and Outputs

16.1 Intended Users

- Individuals analysing social-media comments and replies.
- Users examining workplace messages through the connected API.
- Users who want to spot indirect hostility or potentially threatening language.
- Users looking for constructive alternatives to their original wording.

16.2 Input Types

The application works with text. Supported inputs are single comments, social-media replies, multi-message conversation threads and workplace-related text sent through the workplace API. Image upload is not described as part of the final deployed interface and is not claimed here.

16.3 Output Types

The exact fields shown depend on the selected analysis path, as summarised in Table 16.1.

Table 16.1. Outputs by analysis path

Output	Social-media path	Workplace path
Passive-aggression class	Yes (three classes)	As returned by the workplace API
Model prediction and score	Supervised model output	Response fields of the remote service
Rule evidence and matched spans	Yes	Only if the API returns cue information
Disagreement / review indicator	needs_review	Not applicable
Threat classification	Yes (four classes)	Not part of the documented workplace response
Constructive rewrite	Yes	Not documented
History record	Saved in SQLite where supported	Depends on implementation
Insights and evaluation	Insights page	Not included

17. Model Evaluation and Results

17.1 Evaluation Setup

The detailed README evaluation was run on data/processed/pa_test.csv, which contains 1,382 conversation-grouped held-out samples. The values below are those **reported by the project README**. They were not independently re-run for this report.

17.2 Reported Results

Table 17.1. Reported evaluation of architecture variants on the held-out test set

Architecture variant	Accuracy	Macro F1	Weighted F1	Latency / sample
Hybrid Engine	97.68%	0.9750	0.9771	12.12 ms
Supervised ML	98.70%	0.9872	0.9870	5.47 ms
Rule Engine	40.96%	0.2662	0.2927	0.20 ms
Legacy LinearSVM	25.25%	0.1344	0.1018	0.07 ms

Table 17.2. Other reported results

Item	Reported value	Task
Threat classifier accuracy	99.1%	Four-class threat detection
Threat classifier macro F1	0.963	Four-class threat detection
Automated tests	66 passed, 0 failed	Regression suite

17.3 How to Read These Numbers

The results should not be mixed. The supervised classifier has its own reported performance. The rule engine has a separate, much lower result, and the hybrid engine is a separately reported variant. The threat classifier belongs to a different task altogether. In particular, the rule engine's 40.96% accuracy describes that component alone, measured as a standalone classifier. It is not the performance of the whole application, because the application does not rely on the rule engine alone; its main job is to provide evidence and highlighting.

An earlier, simplified summary in the presentation used different figures and formatting. This report follows the detailed README table, so the supervised accuracy is given as 98.70% and earlier simplified numbers are not combined with it. The hybrid variant scores slightly below the supervised model on accuracy and takes longer per sample, which is a trade-off worth noting: its extra cost buys explainability and disagreement information, not a higher headline score.

17.4 Workplace Model

No evaluation figures for the workplace model are reported here. Its results belong to the separately maintained workplace project and must be taken from that project's own documentation. The illustrative score of 0.948 in Chapter 14 and the latency figure of 2.1 ms mentioned in the presentation are not treated as verified workplace results.

18. Deployment

18.1 Live Application

The presentation identifies the live Streamlit application as hosted on Streamlit Community Cloud at:

https://viknesh2023-wq-passive-aggression-streamlit-app-vtj6mf.streamlit.app/unified_router

The application is reached through a browser. The Unified Router page provides the domain-routing interface. Social-media analysis uses the main project's own inference components, while workplace analysis is requested from the separate FastAPI service over REST.

Source code: https://github.com/viknesh2023-wq/PASSIVE_AGGRESSION_WITH_API

Demo video and presentation:
https://drive.google.com/drive/folders/1elwz1ODxMTES9cOoYEHXepBDUydfO6nO?usp=sharing

18.2 Deployment Status of Components

Table 18.1. Deployment items and their confirmation status

Item	Status in project materials
Streamlit app on Streamlit Community Cloud	Identified in the presentation, with a live URL
Workplace API port	Port 8001 mentioned in the presentation
Docker	Mentioned as a deployment-related technology
Render	Mentioned as a deployment-related technology; successful deployment not confirmed
Public workplace API URL	Not supplied; not stated in this report [To be verified]
Secrets and API base URL	Intended to be set through platform secrets or environment variables, not hardcoded

Because the workplace analysis depends on an external service, the application's workplace feature is only available while that service is running and reachable.

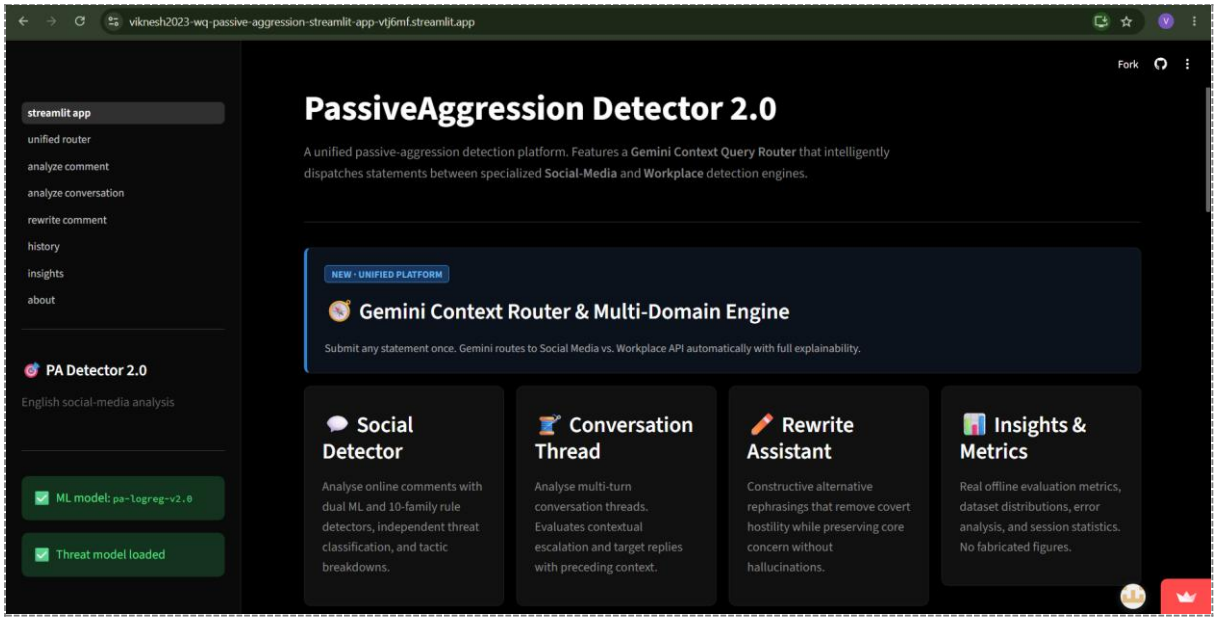


Figure 12. ***Live application deployment.*** The deployed application running in a browser on Streamlit Community Cloud.

19. Ethical Considerations and Limitations

19.1 Ethical Considerations

- The system is meant to assist communication, not to censor it.
- Automated predictions must not be the sole basis for punitive action against anyone.
- Ambiguous statements should be reviewed with human context before drawing conclusions.
- The system tries to separate harmless figurative language from genuine threats, but it can still make mistakes in both directions.
- Constructive rewrites should keep the user's original concern and should not put words or commitments in their mouth.
- Threatening content is never converted into a polite request.

19.2 Limitations

- Passive aggression depends on context, tone and relationship, which text alone does not always reveal.
- Short messages can be ambiguous even for human readers.
- Rule-based detection cannot capture every linguistic variation.
- Model predictions depend on the training data and the quality of its labels. Several sources are auxiliary, and the amount of hand-verified passive-aggression data is small compared with the whole corpus.
- The system handles English only.
- Workplace detection depends on the availability of the external API, and network failures can interrupt it.
- Outputs of the workplace path depend on the implementation of the remote service.
- All results should be interpreted with human judgement.

20. Conclusion and Future Scope

20.1 Conclusion

This project set out to detect a kind of communication that ordinary sentiment tools miss, and to help people respond to it more constructively. It delivers a working Streamlit application that combines a supervised TF-IDF and Logistic Regression classifier with an interpretable ten-family rule engine, keeps both outputs visible, and flags disagreement for human review. Threat detection runs as an independent four-class task, and a rewrite assistant offers calmer wording while refusing to soften real threats.

A context-aware router connects the social-media pipeline with a separately developed workplace service, and REST communication over HTTPX lets the two independently maintained systems cooperate without sharing code. The README reports strong held-out results for the supervised and threat models, and a clearly lower result for the rule engine, which is used mainly for evidence and not as a standalone classifier. The work is a practical, honest prototype with known limits rather than a finished solution to the problem of detecting implied hostility.

20.2 Future Scope

The following are possibilities for later work. None of them should be read as existing functionality.

- Larger and more diverse human-annotated datasets.
- Improved contextual and multilingual detection.
- More robust handling of ambiguous language.
- Further API reliability and monitoring.
- Additional model evaluation across different communication domains.
- Further improvements to explainability.

21. References

References follow IEEE style. Where exact bibliographic details could not be confirmed from the project materials, a verification placeholder is given instead of guessing.

[1] Python Software Foundation, "Python documentation," [Online]. Available: https://docs.python.org/3/

[2] Streamlit Inc., "Streamlit documentation," [Online]. Available: https://docs.streamlit.io

[3] Streamlit Inc., "Streamlit Community Cloud documentation," [Online]. Available: https://docs.streamlit.io/deploy/streamlit-community-cloud

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[5] S. Ramirez, "FastAPI documentation," [Online]. Available: https://fastapi.tiangolo.com

[6] Encode, "Uvicorn documentation," [Online]. Available: https://www.uvicorn.org

[7] Encode, "HTTPX documentation," [Online]. Available: https://www.python-httpx.org

[8] SQLite Consortium, "SQLite documentation," [Online]. Available: https://www.sqlite.org/docs.html

[9] Joblib developers, "Joblib documentation," [Online]. Available: https://joblib.readthedocs.io

[10] GitHub Inc., "GitHub documentation," [Online]. Available: https://docs.github.com

[11] Docker Inc., "Docker documentation," [Online]. Available: https://docs.docker.com

[12] Render, "Render documentation," [Online]. Available: https://render.com/docs

[13] Google, "Gemini API documentation," [Online]. Available: https://ai.google.dev/gemini-api/docs

[14] F. Barbieri, J. Camacho-Collados, L. Neves, and L. Espinosa-Anke, "TweetEval: Unified benchmark and comparative evaluation for tweet classification," in Findings of the Association for Computational Linguistics: EMNLP 2020, 2020.

[15] M. Khodak, N. Saunshi, and K. Vodrahalli, "A large self-annotated corpus for sarcasm," in Proc. 11th Int. Conf. Language Resources and Evaluation (LREC), 2018.

[16] D. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade, and S. Ravi, "GoEmotions: A dataset of fine-grained emotions," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020.

[17] genz_sarc_banter dataset: [Source, authors and year to be verified by the student].

[18] authentic_pa_mined, curated_pa_tactics, contrastive_hard_negatives and gold_pa_anchors: project-constructed sources documented in the project README, available at https://github.com/viknesh2023-wq/PASSIVE_AGGRESSION_WITH_API

[19] SYNPA synthetic dataset: project-generated synthetic dialogues documented in the project README [details to be verified].

[20] G. Viknesh and I. B. Abijit, "PASSIVE_AGGRESSION_WITH_API," project README and presentation, [Online]. Available: https://github.com/viknesh2023-wq/PASSIVE_AGGRESSION_WITH_API